



RAPPORT

APPROCHES SUPERVISÉES ET NON SUPERVISÉES

Aymen Rguieg

Introduction à l'IA : Travaux pratiques

13 Septembre 2024

Pour une meilleure visualisation des schémas et du code, veuillez consulter le lien suivant sur Google Colab
: <https://colab.research.google.com/drive/1qzUSTUxPMifm5C187WgAcsW8WG2Z8kwP?usp=sharing>.

Abstract

This report examines the application of supervised and unsupervised learning algorithms to a colon cancer gene expression dataset. The objective is to classify samples as either normal or cancerous using methods such as logistic regression, SVM, and k-NN, following steps like data cleaning, training, and testing. Additionally, we investigate feature importance and evaluate model performance using accuracy and other relevant metrics.

Résumé

Ce rapport examine l'application d'algorithmes d'apprentissage supervisé et non supervisé sur un jeu de données d'expression génétique du cancer du côlon. L'objectif est de classer les échantillons en tant que normaux ou cancéreux en utilisant des méthodes telles que la régression logistique, SVM, et k-NN, après des étapes de nettoyage des données, d'entraînement et de test. De plus, nous analysons l'importance des caractéristiques et évaluons les performances des modèles en fonction de l'exactitude et d'autres métriques pertinentes.

Contents

1	Introduction	3
2	Préparation du dataset	3
2.1	Chargement des données	3
2.2	Nettoyage les données	4
3	Exploration des données	4
3.1	Type de colonne	5
3.2	Distribution des classes	5
3.3	Analyse statistique descriptive	7
4	Application des algorithmes de classification	9
4.1	Séparation des features (gènes) et la variable cible (normal/cancer). . .	9
4.2	Division des données en ensembles d'entraînement et de tests et standardisation	10
4.3	Application des algorithmes de classification	11
4.3.1	Régression logistique	11
4.3.2	SVM (Support Vector Machine)	14
4.3.3	k-NN (k-Nearest Neighbors)	16
4.3.4	Random Forest Classifier	18
5	L'impact de chaque gène dans le modèle LR	20
6	Evaluation de la performance des deux modèles en exploitant le top 1,2 et 3 des features.	22
7	Comparaison entre les modèles appliqués sur colon_data et la nouvelle data. .	23
8	Case study	24
9	Conclusion	24

1 Introduction

Nous avons reçu des jeux de données comprenant des profils d'expression génétique de plusieurs patients. L'objectif principal est d'identifier si un patient est atteint de cancer en se basant sur ces profils d'expression génétique. Le jeu de données contient à la fois des échantillons cancéreux et non cancéreux, ce qui nous permet d'entraîner et d'évaluer divers modèles d'apprentissage automatique.

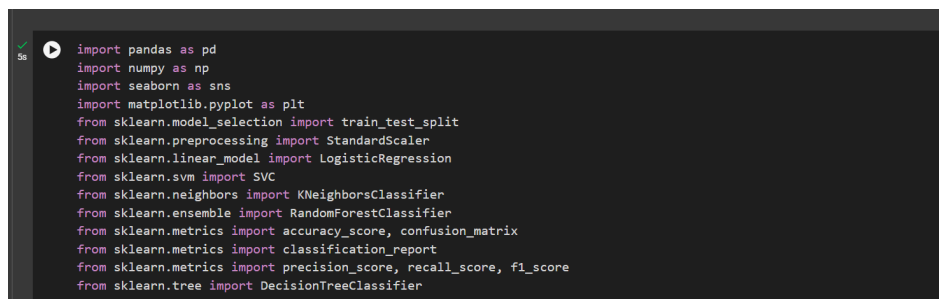
2 Préparation du dataset

2.1 Chargement des données

Nous avons commencé par télécharger le jeu de données "**colon_cancer**". Dès que nous l'examinons, nous remarquons les points suivants :

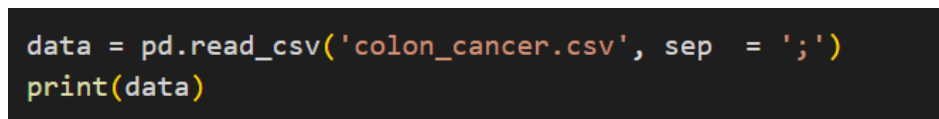
- Il s'agit d'un jeu de données relativement petit, avec environ 804 lignes et 62 colonnes, ce qui le rend facile à gérer.
- La dernière colonne, "**tissue_status**", est la variable cible et peut prendre deux valeurs: *normal* ou *tumour*. La présence de seulement deux classes simplifie quelque peu notre étude.

Nous avons ensuite ouvert un notebook Python et chargé les données en utilisant les bibliothèques Pandas et NumPy. Pour commencer, nous avons importé les bibliothèques requises avec les commandes `import pandas as pd` et `import numpy as np`. Puis, nous avons employé la fonction `pd.read_csv()` avec l'argument `sep=;` pour lire le fichier CSV du jeu de données et le stocker dans un DataFrame Pandas, facilitant ainsi la manipulation et l'analyse des données.



```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.metrics import classification_report
from sklearn.metrics import precision_score, recall_score, f1_score
from sklearn.tree import DecisionTreeClassifier
```

Figure 1: Les bibliothèques



```
data = pd.read_csv('colon_cancer.csv', sep = ';')
print(data)
```

Figure 2: Chargement de la data

Le paramètre `sep=;` joue un rôle crucial. Il nous a permis de lire correctement les données en séparant les colonnes délimitées par des points-virgules (;).

2.2 Nettoyage les données

Avant d'entraîner et de tester notre modèle, il est essentiel de procéder à un nettoyage rigoureux des données. Cette étape consiste à s'assurer qu'il n'y a ni valeurs manquantes ni doublons, ce qui pourrait fausser les résultats de notre analyse. Pour ce faire, nous avons vérifié la présence de valeurs nulles et supprimé les entrées redondantes afin de garantir que notre jeu de données soit à la fois complet et unique. Ce processus de nettoyage est fondamental pour améliorer la précision et la fiabilité du modèle.

```
total_missing_values = data.isnull().sum().sum()
print(f"Total des valeurs manquantes : {total_missing_values}")
```

Figure 3: Calcule de la somme des valeurs manquantes

Le code nous indique qu'**aucune valeur manquante** n'est présente dans les données.

```
duplicate_count = data.duplicated().sum()
print(f"Nombre de doublons : {duplicate_count}")
```

Figure 4: Calcule de la somme des doublons

Le code confirme qu'**aucun doublon** n'est présent dans les données.

Il n'est pas nécessaire de nettoyer les données, car elles sont déjà propres et prêtes à être utilisées. Nous avons vérifié l'intégrité des données et confirmé qu'elles ne contiennent ni valeurs manquantes ni doublons. Ainsi, nous pouvons procéder directement à l'analyse sans avoir besoin de nettoyage supplémentaire.

3 Exploration des données

Pour analyser notre problème, nous avons d'abord créé une liste de noms de colonnes. Bien que cela puisse sembler inutile, cela nous a aidés à mieux comprendre les données que nous manipulions, comme le montre la figure 5.

```
column_names = data.columns.tolist()
print(column_names)
```

Figure 5: Nom des colonnes

3.1 Type de colonne

En utilisant `data.info()` on peut visualiser le type des 61 colonnes comme le montre la figure 6

```
data.info()
```

RangeIndex: 804 entries, 0 to 803			
Data columns (total 62 columns):			
#	Column	Non-Null Count	Dtype
0	id_sample	804 non-null	object
1	ADH1C	804 non-null	float64
2	DHRS11	804 non-null	float64
3	UGP2	804 non-null	float64
4	SLC7A5	804 non-null	float64
5	CTSS	804 non-null	float64
6	DAO	804 non-null	float64
7	NIBAN1	804 non-null	float64
8	PRUNE2	804 non-null	float64
9	FOXF2	804 non-null	float64
10	TENT5C	804 non-null	float64
11	KLF10	804 non-null	float64
12	FABP1	804 non-null	float64
13	RPSAP19	804 non-null	float64
14	NCAPH	804 non-null	float64
15	TPM1	804 non-null	float64
16	PLA2G12B	804 non-null	float64
17	PLAAT4	804 non-null	float64
18	IGLV8-61	804 non-null	float64
19	GSS	804 non-null	float64
20	L1TD1	804 non-null	float64
21	RNF186	804 non-null	float64
22	HES2	804 non-null	float64
23	MXRA8	804 non-null	float64
24	SOX18	804 non-null	float64
25	NDFIP2	804 non-null	float64

Figure 6: Les infos de la data

On peut clairement constater que 60 colonnes (gènes) sont de type float, tandis que les deux autres colonnes, `id_sample` et `Tissue_status`, sont de types différents.

3.2 Distribution des classes

L'un des principaux problèmes en apprentissage automatique est le **déséquilibre des classes**, qui peut entraîner un biais du modèle en faveur de la classe majoritaire. Cela peut conduire à une précision trompeuse et à une évaluation insuffisante des performances sur la classe minoritaire, affectant ainsi la capacité du modèle à généraliser et à prendre des décisions équilibrées, particulièrement dans des contextes critiques.

D'où la **nécessité** de **visualiser la distribution des classes** et de connaître le nombre de classes disponibles au préalable.

```

class_counts = data['tissue_status'].value_counts()
print("Nombre de classes :", len(class_counts))
print("Noms des classes :", class_counts.index.tolist())

```

```

Nombre de classes : 2
Noms des classes : ['normal', 'tumoral']

```

Figure 7: Les classes du dataset

On constate qu'il y'a que deux classes qui sont normal et tumoral.

```

sns.countplot(x='tissue_status', data=data)
plt.title('Distribution des classes (Cancer vs Normal)')
plt.xlabel('Classe (0 = Normal, 1 = Cancer)')
plt.ylabel('Nombre d\'échantillons')
plt.show()

```

```

class_counts = data['tissue_status'].value_counts()
plt.figure(figsize=(8, 6))
plt.pie(class_counts, labels=class_counts.index, autopct='%1.1f%%', startangle=90)
plt.title('Distribution des classes')
plt.show()

```

Figure 8: Visualisations sur la distribution des classes

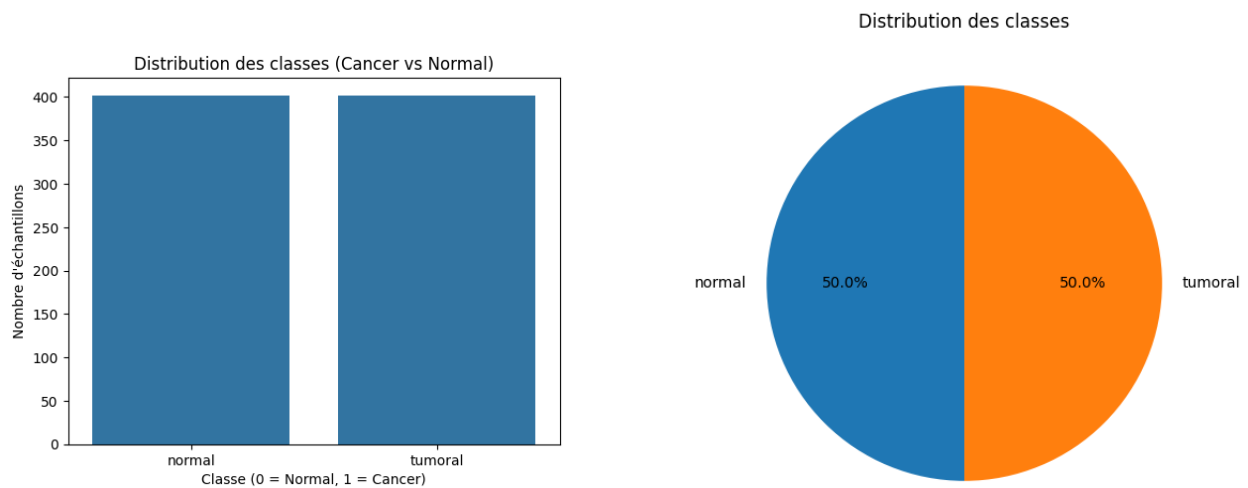


Figure 9: Les graphs de visualisation

Les données sont parfaitement équilibrées, avec une répartition équitable des classes.

3.3 Analyse statistique descriptive

```
print(data.describe())
```

Statistiques descriptives des gènes :							
	ADH1C	DHRS11	UGP2	SLC7A5	CTSS	DAO	\
count	804.000000	804.000000	804.000000	804.000000	804.000000	804.000000	
mean	5.588943	4.354817	5.557694	4.513570	6.717535	7.218184	
std	2.889901	1.431179	1.150565	1.614450	1.077821	1.411969	
min	0.610504	1.427879	2.027201	1.685024	3.668601	1.164886	
25%	3.174250	3.118844	4.611060	3.030022	6.065937	6.241714	
50%	5.467024	4.125990	5.412284	4.382953	6.762908	7.578782	
75%	8.444756	5.696566	6.556969	5.918848	7.485487	8.263561	
max	10.263846	7.022241	7.571114	8.059430	8.990703	10.079135	

	NIBAN1	PRUNE2	FOXF2	TENTSC	...	SYTL2	\
count	804.000000	804.000000	804.000000	804.000000	...	804.000000	
mean	2.737937	2.806573	3.064394	3.658318	...	3.599294	
std	1.466906	1.243826	1.191486	1.258059	...	1.054504	
min	0.716737	0.368200	0.610504	0.858074	...	1.286220	
25%	1.550382	1.864561	2.198244	2.655091	...	2.878356	
50%	2.297732	2.741509	2.900393	3.605686	...	3.613342	
75%	3.669509	3.574304	3.752899	4.686086	...	4.417632	
max	6.682541	6.591924	6.276628	6.350919	...	5.713776	

	BSPRY	CDHR2	ERRFI1	CLIC5	PLLP	GAL	\
count	804.000000	804.000000	804.000000	804.000000	804.000000	804.000000	
mean	4.682718	4.225814	4.575397	4.097244	1.823268	2.822931	
std	0.917027	1.656088	0.859138	1.276589	0.721587	1.338349	
min	1.868389	0.637047	2.250347	1.015235	0.118945	0.610504	
25%	4.349295	2.921069	3.958903	3.131267	1.337715	1.840002	
50%	4.869313	4.116251	4.537381	3.975276	1.689671	2.591044	
75%	5.249372	5.507864	5.070072	5.118110	2.211967	3.661012	
max	7.029946	9.045672	7.647284	7.129499	4.763212	7.545757	

Figure 10: Description statistique de la data

On remarque que les moyennes sont relativement petit. De meme pour les variances. ce qui pose la question suivante : **Est-il nécessaire de standardiser et de normaliser notre data?** Ca nous savons tous que le but de la standardisation et de la normalisation est d'ajuste les données pour avoir une moyenne de 0 et un écart type de 1 et de les transforme pour les ramener dans une plage spécifique, souvent [0, 1], afin d'améliorer la performance des algorithmes de machine learning.

```
gene_columns = data.columns[1:-1]
selected_genes = gene_columns[:10]

for gene in selected_genes:
    plt.figure(figsize=(8, 6))
    plt.hist(data[gene], bins=20)
    plt.title(f'Histogramme de {gene}')
    plt.xlabel(gene)
    plt.ylabel('Fréquence')
    plt.show()
```

Figure 11: Les 10 premières colonnes de données de gènes, visualisant la distribution des valeurs de chaque gène sélectionné.

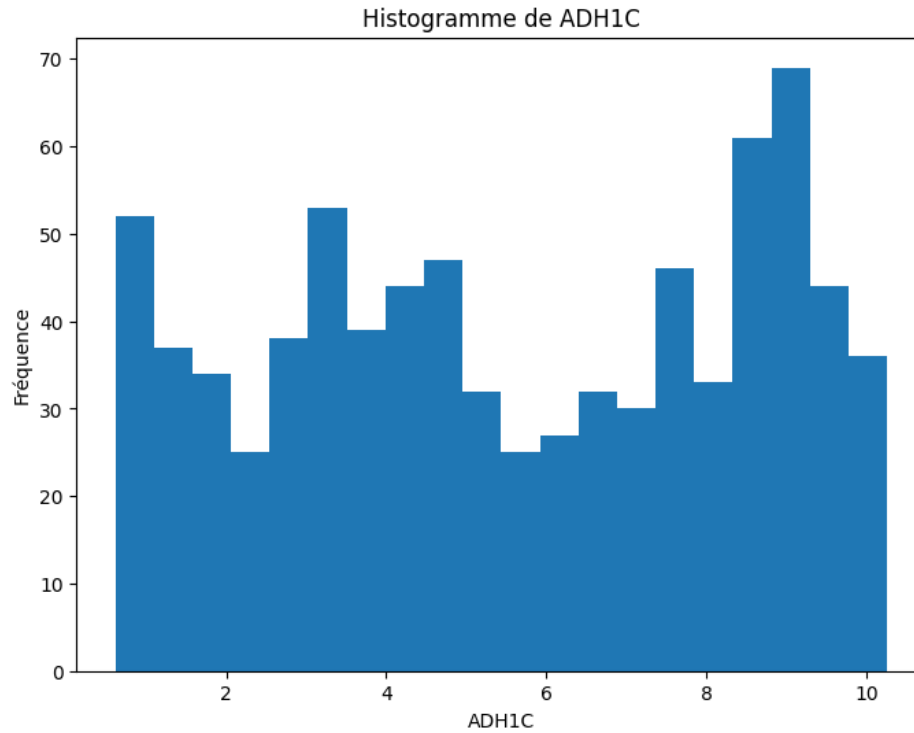


Figure 12: Exemple d'histogramme pour le gène ADH1C

Ces graphiques permettent de visualiser la fréquence des valeurs numériques associées à chaque gène.

```
plt.figure(figsize=(12, 8))
data[selected_genes].boxplot()
plt.title('Diagramme de Boîte pour les Gènes Sélectionnés')
plt.xlabel('Gènes')
plt.ylabel('Expression Gène')
plt.xticks(rotation=45)
plt.show()
```

Figure 13: code de diagrammes en boîte

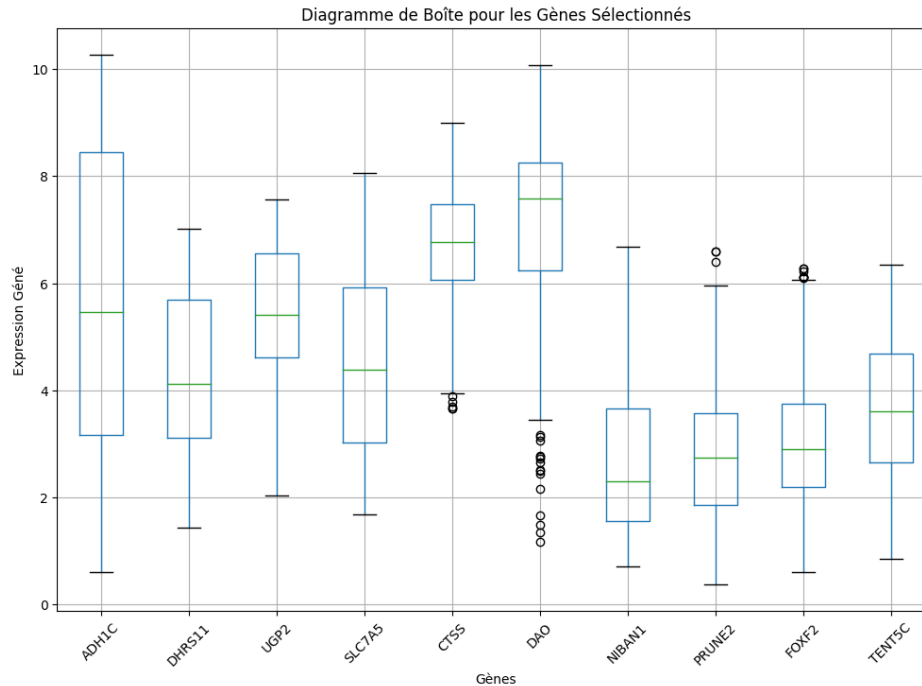


Figure 14: diagrammes en boîte

Le diagramme de boîte présenté ci-dessus illustre la distribution des niveaux d'expression de différents gènes sélectionnés. Chaque boîte correspond à un gène et montre l'étendue des valeurs d'expression de ce gène, avec la ligne verte représentant la médiane. Les limites de la boîte indiquent le premier et le troisième quartile (Q1 et Q3), tandis que les "whiskers" (lignes horizontales) s'étendent jusqu'aux valeurs minimales et maximales non considérées comme des valeurs aberrantes. Les points noirs autour de certains gènes, comme CTSS et DAO, représentent des valeurs aberrantes, c'est-à-dire des observations qui s'écartent significativement des autres données. Ce diagramme permet de visualiser la variabilité des données d'expression des gènes, d'identifier les gènes présentant une distribution d'expression plus large ou plus restreinte, ainsi que de détecter d'éventuelles anomalies ou valeurs extrêmes dans l'ensemble de données analysé.

4 Application des algorithmes de classification

4.1 Séparation des features (gènes) et la variable cible (normal/cancer).

Avant de former notre modèle, il est essentiel de sélectionner les caractéristiques pertinentes (features) et la colonne cible. Cette étape garantit que seules les données indispensables sont utilisées pour l'entraînement, améliorant ainsi les performances du modèle et assurant un apprentissage efficace et précis.

```
X = data.drop(['tissue_status', 'id_sample'], axis=1)
# Iterate over the columns of the new DataFrame 'X'
for col in X.columns:
    if X[col].dtype == 'object':
        X[col] = pd.to_numeric(X[col], errors='coerce')
```

Figure 15: Features

```
Y = data['tissue_status']
```

Figure 16: Colonne cible

Nous remarquons que la colonne cible est de type object donc nous allons opter pour un encodage simple pour la transformer en une colonne numérique comme l'indique la figure 17

```
Y = Y.apply(lambda x: 1 if x == "normal" else 0)
print(Y)
```

Figure 17: Encodage simple

4.2 Division des données en ensembles d'entraînement et de tests et standardisation

```
X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2, random_state=42, stratify=Y)
```

Figure 18: Division des données

- La proportion du jeu de données original qui sera allouée au sous-ensemble de test. Ici, 0.2 signifie que 20% des données seront utilisées pour le test, et 80% pour l'entraînement.
- Le paramètre **random_state** fixe une graine pour le générateur de nombres aléatoires, garantissant ainsi que la division des données est reproductible et donne les mêmes résultats à chaque exécution.

```

scaler = StandardScaler()
scaler.fit(X_train)

X_train_scaled = scaler.transform(X_train)

X_test_scaled = scaler.transform(X_test)

print(f"Train set: {X_train_scaled.shape}, Test set: {X_test_scaled.shape}")

Train set: (643, 60), Test set: (161, 60)

```

Figure 19: Standardisation des données

- Le **StandardScaler** est utilisé pour centrer et réduire les données en leur appliquant une transformation de normalisation. Il soustrait la moyenne des données et divise par l'écart type, de sorte que les données normalisées aient une moyenne de 0 et un écart type de 1.

$$X_{\text{scaled}} = \frac{X - \mu}{\sigma}$$

- X : La valeur initiale d'une caractéristique (feature) dans le jeu de données.
- μ : La moyenne des valeurs de cette caractéristique dans le jeu de données d'entraînement.
- σ : L'écart type des valeurs de cette caractéristique dans le jeu de données d'entraînement.
- La méthode **fit()** calcule la moyenne et l'écart type de X_{train} , le jeu de données d'entraînement.
- La méthode **transform()** utilise la moyenne et l'écart type calculés à partir de X_{train} et X_{test} pour standardiser les données d'entraînement et de test.

Nous divisons d'abord les données en ensembles d'entraînement et de test pour éviter de mélanger les informations entre eux. Si nous standardisons les données avant de les diviser, la moyenne et l'écart type utilisés pour ajuster les données seraient calculés à partir de toutes les données, y compris celles de l'ensemble de test. Cela donnerait au modèle une connaissance indirecte des données de test, faussant ainsi l'évaluation de ses performances. En divisant les données avant de standardiser, nous calculons la moyenne et l'écart type uniquement sur l'ensemble d'entraînement. Ensuite, ces mêmes statistiques sont appliquées à l'ensemble de test. De cette façon, l'ensemble de test reste inconnu du modèle jusqu'à ce que l'on teste sa performance finale, garantissant une évaluation juste et précise de sa capacité à généraliser sur de nouvelles données.

4.3 Application des algorithmes de classification

4.3.1 Régression logistique

La régression logistique est une méthode statistique utilisée pour modéliser la relation entre une variable dépendante binaire (comme "succès" ou "échec") et une ou plusieurs variables in-

dépendantes, en estimant la probabilité d'occurrence de l'un des deux résultats possibles.

La formule générale pour la combinaison linéaire dans une régression logistique est :

$$f(X) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n$$

où :

- $f(X)$ est la combinaison linéaire des variables indépendantes,
- β_0 est l'ordonnée à l'origine (interception),
- $\beta_1, \beta_2, \dots, \beta_n$ sont les coefficients de régression,
- $X_1, X_2, \dots, X_n$ sont les variables indépendantes.

Pour obtenir la probabilité $\hat{p}$ de la classe 1, on applique la fonction sigmoïde à $f(X)$:

$$\hat{p} = \frac{1}{1 + e^{-f(X)}}$$

```
y_test = y_test.astype('category').cat.codes
y_train = y_train.astype('category').cat.codes
model_lr = LogisticRegression()
model_lr.fit(X_train, y_train)
y_train_pred_lr = model_lr.predict(X_train)
y_test_pred_lr = model_lr.predict(X_test)
accuracy_train = accuracy_score(y_train, y_train_pred_lr)
print("Précision sur l'ensemble d'entraînement (Training):", accuracy_train)
accuracy_test = accuracy_score(y_test, y_test_pred_lr)
print("Précision sur l'ensemble de test:", accuracy_test)
```

Figure 20: Régression logistique

```
Précision sur l'ensemble d'entraînement (Training): 1.0
Précision sur l'ensemble de test: 1.0
```

Figure 21: Accuracy

L'accuracy d'entraînement (ou précision d'entraînement) mesure la performance du modèle sur **le même ensemble** de données utilisé pour l'entraîner. Elle indique à quel point le modèle a appris à prédire correctement les étiquettes des exemples qu'il a déjà vus. Une précision d'entraînement élevée peut signifier que le modèle s'adapte bien aux données d'entraînement, mais elle peut aussi indiquer un surapprentissage (overfitting) si le modèle est trop complexe et s'ajuste trop aux particularités de ces données.

En revanche, **l'accuracy de test** (ou précision de test) évalue **la performance du modèle sur un ensemble de données inédites** (les données de test) qui n'ont pas été

utilisées pendant l'entraînement. Elle est plus représentative de la capacité du modèle à généraliser ses prédictions à de nouvelles données. Si l'accuracy de test est significativement plus faible que l'accuracy d'entraînement, cela peut indiquer que le modèle ne généralise pas bien et qu'il a probablement surappris les données d'entraînement.

Une précision de 1.0 sur l'ensemble d'entraînement signifie que le modèle a parfaitement prédit toutes les étiquettes sur les données qu'il a utilisées pour apprendre, tandis qu'une précision de 1.0 sur l'ensemble de test indique qu'il a également prédit sans erreur sur les nouvelles données qu'il n'a jamais vues auparavant. Ce résultat suggère que le modèle est non seulement très bien ajusté aux données d'entraînement, mais aussi qu'il **généralise parfaitement aux données de test**, ce qui peut être rare dans des situations réelles.

```
print("\nPrécision du modèle :")
print(precision_score(y_test, y_test_pred_lr))
print("\nRappel du modèle :")
print(recall_score(y_test, y_test_pred_lr))
print("\nF1-score du modèle :")
print(f1_score(y_test, y_test_pred_lr))
```

```
Précision du modèle :
1.0
```

```
Rappel du modèle :
1.0
```

```
F1-score du modèle :
1.0
```

Figure 22: Précision-recall et F1-score

Une précision de 1.0, un rappel de 1.0 et un F1-score de 1.0 indiquent que le modèle a une performance parfaite.

- **Précision (precision) de 1.0** signifie que le modèle n'a fait aucune fausse prédiction positive : chaque fois qu'il a prédit une classe positive, il avait raison.
- **Rappel (recall) de 1.0** indique que le modèle a correctement identifié toutes les instances positives du jeu de données : il n'a manqué aucune instance appartenant à la classe positive.
- **F1-score de 1.0** (qui est la moyenne harmonique de la précision et du rappel) confirme cet équilibre parfait entre précision et rappel.

Ensemble, ces scores de 1.0 montrent que le modèle fonctionne de manière idéale, sans aucune erreur de classification. Toutefois, dans la pratique, un résultat aussi parfait peut parfois être suspect, car il pourrait indiquer un modèle surajusté ou un jeu de données trop simple ou biaisé.

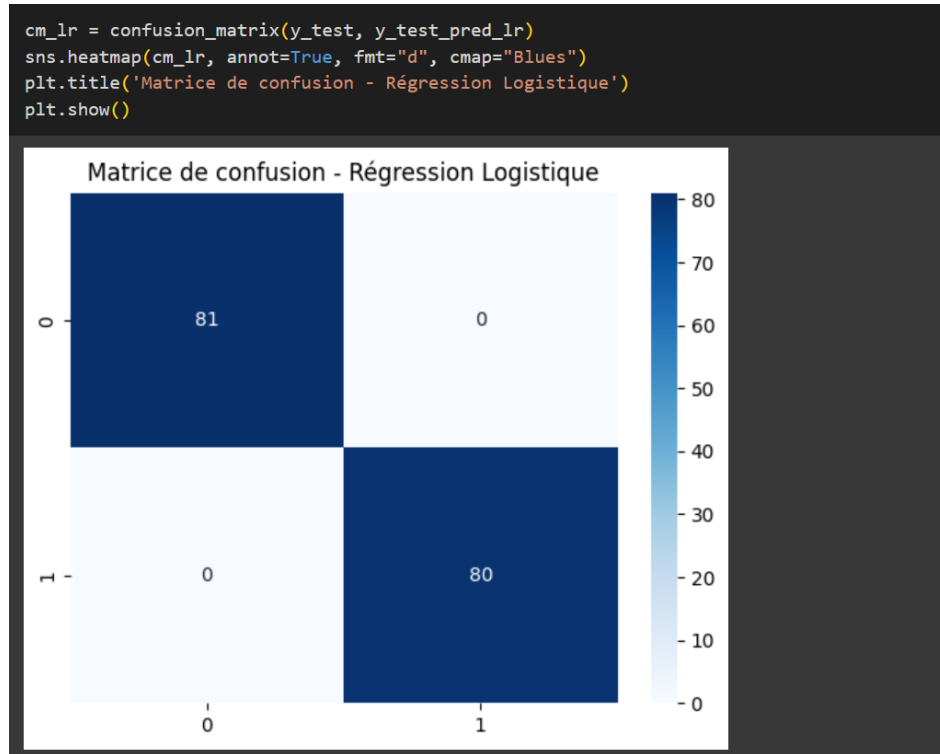


Figure 23: Matrice de confusion

La matrice de confusion montre 80 vrais positifs (True Positives) et 80 vrais négatifs (True Negatives), ce qui indique que le modèle a correctement prédit 80 instances positives et 80 instances négatives. Cette performance suggère que le modèle est efficace pour identifier les deux classes correctement, sans confondre les instances positives avec des négatives ni les instances négatives avec des positives.

4.3.2 SVM (Support Vector Machine)

Les Support Vector Machines (SVM) sont des algorithmes de classification qui cherchent à trouver l'**hyperplan optimal** séparant les classes avec la **plus grande marge possible**. Pour des données linéairement séparables, l'hyperplan est une ligne ou un plan, tandis que pour des données non linéaires, les SVM utilisent des noyaux pour transformer les données dans un espace de dimensions plus élevées, permettant ainsi une séparation linéaire. Cette approche permet aux SVM de gérer des situations complexes et de fournir des classifications précises.

```

model_svm = SVC()
model_svm.fit(X_train, y_train)
y_train_pred_svm = model_svm.predict(X_train)
y_test_pred_svm = model_svm.predict(X_test)
accuracy_train_svm = accuracy_score(y_train, y_train_pred_svm)
print("Précision sur l'ensemble d'entraînement (Training) SVM :", accuracy_train_svm)
accuracy_test_svm = accuracy_score(y_test, y_test_pred_svm)
print("Précision sur l'ensemble de test SVM :", accuracy_test_svm)

Précision sur l'ensemble d'entraînement (Training) SVM : 1.0
Précision sur l'ensemble de test SVM : 1.0

```

Figure 24: SVM

Les précisions de 1.0 sur les ensembles d'entraînement et de test indiquent que le modèle SVM a réussi à prédire correctement toutes les instances dans les deux ensembles de données, suggérant une performance parfaite sans erreur de classification.

```

print("\nPrécision du modèle :")
print(precision_score(y_test, y_test_pred_svm))
print("\nRappel du modèle :")
print(recall_score(y_test, y_test_pred_svm))
print("\nF1-score du modèle :")
print(f1_score(y_test, y_test_pred_svm))

Précision du modèle :
1.0

Rappel du modèle :
1.0

F1-score du modèle :
1.0

```

Figure 25: Précision-recall et F1-score

Les scores parfaits de précision, rappel et F1 indiquent que le modèle a atteint une performance idéale, avec une prédiction correcte à 100 % pour toutes les classes.

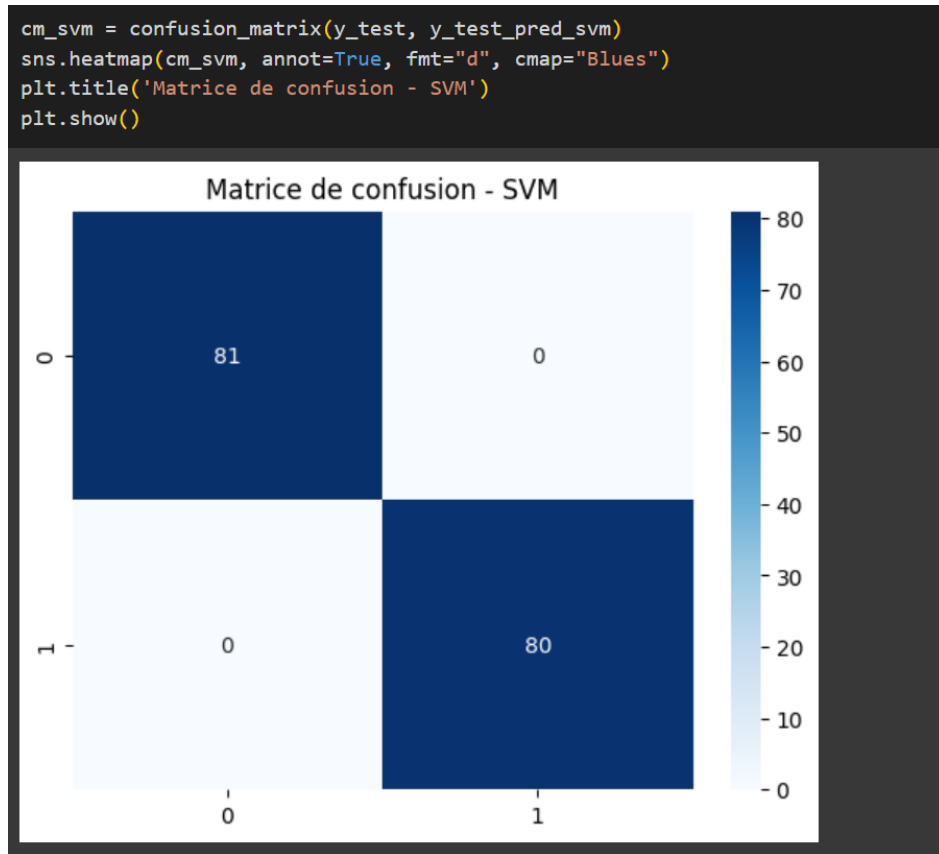


Figure 26: Matrice de confusion

La matrice de confusion montre 80 vrais positifs et 80 vrais négatifs, indiquant que le modèle SVM a correctement prédit toutes les instances positives et négatives, démontrant ainsi une classification efficace des deux classes.

Dans notre cas, les performances des SVM et de la régression logistique sont identiques, avec des scores parfaits sur les ensembles d'entraînement et de test, indiquant que les deux modèles fonctionnent de manière équivalente sur les données fournies.

4.3.3 k-NN (k-Nearest Neighbors)

Le k-Nearest Neighbors (k-NN) est un algorithme de classification qui classe un point de données en fonction de la classe de ses k voisins les plus proches. Pour prédire la classe d'un point, l'algorithme recherche les k points de données les plus proches dans l'ensemble d'entraînement (généralement mesurés par la distance euclidienne) et attribue la classe la plus fréquente parmi eux. Simple et efficace pour des petits ensembles de données, le k-NN est sensible au choix de k et aux données bruitées.

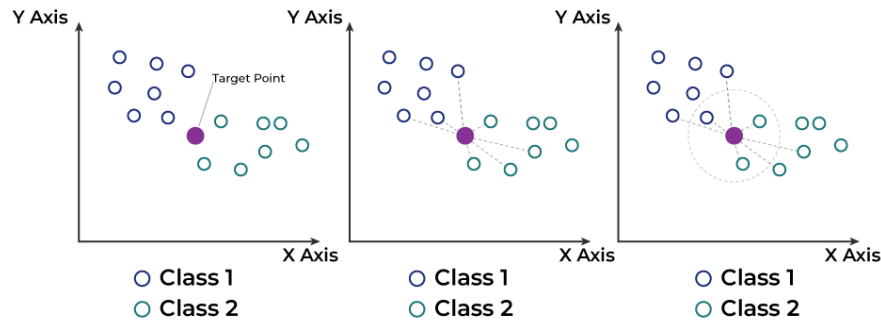


Figure 27: k-Nearest Neighbors

comment choisir la valeur k ?

Pour choisir la valeur de k dans l'algorithme k-NN, il est recommandé de tester plusieurs valeurs en utilisant la validation croisée pour identifier celle qui donne la meilleure précision. En général, une petite valeur de k (comme 1 ou 3) peut rendre le modèle sensible au bruit, tandis qu'une grande valeur de k peut le rendre trop général. Un bon point de départ est de tester des valeurs impaires (pour éviter les égalités) et choisir k qui minimise l'erreur de classification.

```
from sklearn.model_selection import GridSearchCV
param_grid = {'n_neighbors': range(1, 21)}
knn = KNeighborsClassifier()
grid = GridSearchCV(knn, param_grid, cv=5, scoring='accuracy')
grid.fit(X_train, y_train)
print("Meilleure valeur pour k:", grid.best_params_['n_neighbors'])
print("Meilleure précision:", grid.best_score_)

Meilleure valeur pour k: 1
Meilleure précision: 0.9984375
```

Figure 28: Validation croisée

Ce code utilise la méthode **GridSearchCV** de scikit-learn pour trouver la meilleure valeur de k dans un modèle de k-Nearest Neighbors (k-NN). Il définit une grille de paramètres (param_grid) avec des valeurs de k allant de 1 à 20 et configure une recherche de grille avec validation croisée à 5 folds (cv=5) pour évaluer la précision du modèle. En entraînant le modèle sur les données d'entraînement (X_train, y_train), GridSearchCV teste chaque valeur de k et identifie celle qui maximise la précision moyenne (scoring='accuracy'). Enfin, le code affiche la meilleure valeur de k (grid.best_params_['n_neighbors']) et la précision correspondante (grid.best_score_), offrant ainsi une solution optimisée pour le modèle k-NN.

```

n_neighbors = 1
model_knn = KNeighborsClassifier(n_neighbors=n_neighbors)
model_knn.fit(X_train, y_train)
y_train_pred_knn = model_knn.predict(X_train)
y_test_pred_knn = model_knn.predict(X_test)
accuracy_train_knn = accuracy_score(y_train, y_train_pred_knn)
print("Précision sur l'ensemble d'entraînement (Training) k-NN :", accuracy_train_knn)
accuracy_test_knn = accuracy_score(y_test, y_test_pred_knn)
print("Précision sur l'ensemble de test k-NN :", accuracy_test_knn)

Précision sur l'ensemble d'entraînement (Training) k-NN : 1.0
Précision sur l'ensemble de test k-NN : 1.0

```

Figure 29: K-NN

Le modèle k-NN a obtenu une précision parfaite de 1.0 sur l'ensemble d'entraînement, indiquant qu'il a correctement classé toutes les données d'entraînement. De même, une précision de 1.0 sur l'ensemble de test montre que le modèle a également prédit toutes les données de test sans erreur.

4.3.4 Random Forest Classifier

Le Random Forest Classifier est un algorithme de machine learning qui utilise un ensemble de nombreux arbres de décision pour effectuer des prédictions. Chaque arbre est entraîné sur un échantillon aléatoire des données, et ses décisions sont combinées (par vote majoritaire) pour donner la prédiction finale. Cette méthode réduit le risque de surapprentissage (overfitting) en moyennant les erreurs des arbres individuels, ce qui la rend plus robuste et précise que l'utilisation d'un seul arbre de décision. Le Random Forest est efficace pour gérer des ensembles de données complexes, avec de nombreuses variables, et est souvent utilisé pour des tâches de classification et de régression.

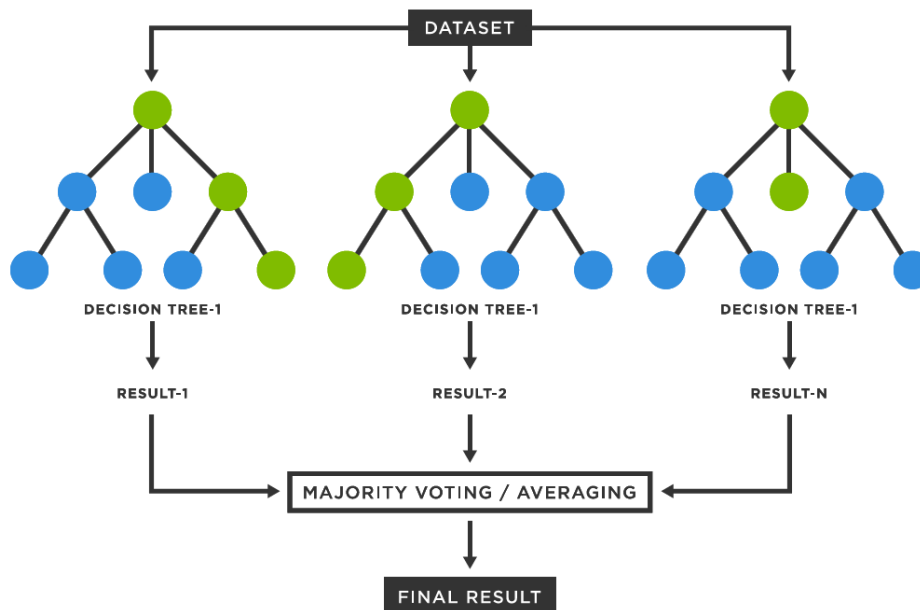


Figure 30: Random forest

```

model_rf = RandomForestClassifier(n_estimators=100)
model_rf.fit(X_train, y_train)
y_train_pred_rf = model_rf.predict(X_train)
y_test_pred_rf = model_rf.predict(X_test)
accuracy_train_rf = accuracy_score(y_train, y_train_pred_rf)
print("Précision sur l'ensemble d'entraînement (Training) Random Forest :", accuracy_train_rf)
accuracy_test_rf = accuracy_score(y_test, y_test_pred_rf)
print("Précision sur l'ensemble de test Random Forest :", accuracy_test_rf)

Précision sur l'ensemble d'entraînement (Training) Random Forest : 1.0
Précision sur l'ensemble de test Random Forest : 1.0

```

Figure 31: code de random forest

Le paramètre **n_estimators** spécifie combien d'arbres de décision individuels seront entraînés pour constituer la forêt. Dans cet exemple, **n_estimators=100** signifie que le modèle entraînera 100 arbres de décision distincts.

Le modèle de forêt aléatoire montre une excellente capacité de généralisation avec une précision de 99,38 % sur l'ensemble de test, mais la précision parfaite de 100 % sur l'ensemble d'entraînement pourrait indiquer un possible surapprentissage.

```

importances = model_rf.feature_importances_
feature_importances = pd.DataFrame({'Feature': X.columns, 'Importance': importances})
feature_importances = feature_importances.sort_values('Importance', ascending=False)
top_8_features = feature_importances.head(8)
plt.figure(figsize=(10, 6))
sns.barplot(x='Importance', y='Feature', data=top_8_features)
plt.title('Top 8 des caractéristiques importantes (Random Forest)')
plt.xlabel('Importance')
plt.ylabel('Caractéristiques')
plt.show()
print(top_8_features)

```

Figure 32: Les caractéristiques les plus importantes

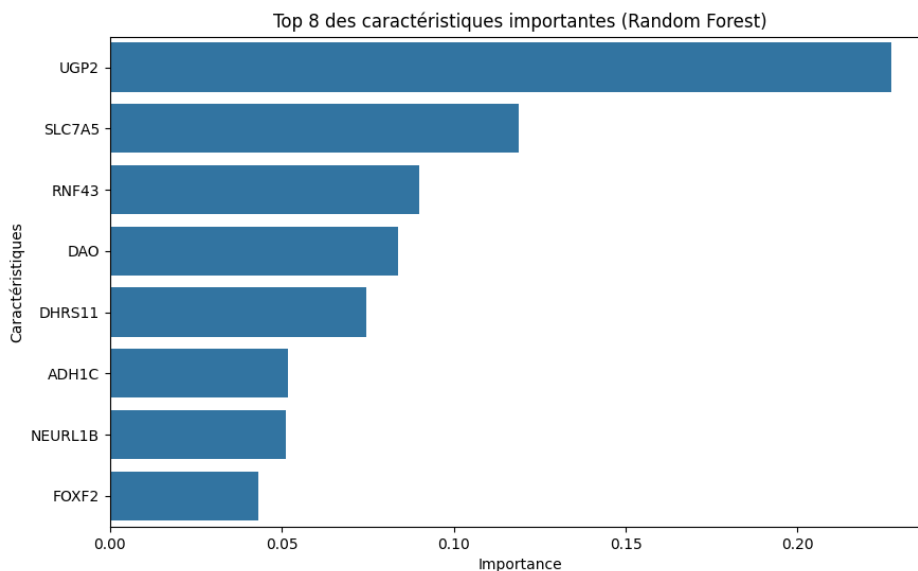


Figure 33: Graphe

Le graphique montre les 8 caractéristiques les plus importantes identifiées par le modèle de forêt aléatoire. La caractéristique UGP2 est la plus significative pour la prédiction, suivie par SLC7A5 et RNF43, indiquant leur contribution élevée à la performance du modèle.

5 L'impact de chaque gène dans le modèle LR

Dans cette section, nous allons tenter de déterminer les coefficients de la régression linéaire (RL) afin d'identifier les gènes qui exercent le plus d'influence. Un coefficient élevé, en valeur absolue, indique que le gène associé a un impact significatif sur l'ensemble des données. En d'autres termes, plus le coefficient est éloigné de zéro, plus il contribue à expliquer la variabilité dans notre modèle. L'objectif est donc de repérer les gènes dont les coefficients de régression sont les plus élevés, car ils représentent des facteurs clés influençant le phénomène étudié.

```
coefficients_lr = model_lr.coef_
print("\nCoefficients de la Régression Logistique :")
print(coefficients_lr)

Coefficients de la Régression Logistique :
[[ 0.38606688  0.30986872  0.59660495 -0.89527506  0.03439969  0.56786278
  0.45675903 -0.01617665  0.47035906  0.25015267  0.23483311  0.20272474
  0.13017926 -0.48393011  0.23144088  0.23012128  0.34583982  0.14013403
  0.19641454 -0.05265177  0.00146101 -0.1526326  -0.08821156  0.2723994
 -0.10321118  0.1034014  0.54255293 -0.47795021  0.12489404  0.06356193
 -0.14140745  0.30101377  0.01620483  0.26318078 -0.1101401  -0.00270628
  0.09443089  0.15895714  0.02810876 -0.04836053 -0.26195017  0.16254502
 -0.09371791 -0.02876838 -0.81425345 -0.32845314 -0.21791278  0.26035621
 -0.16923938 -0.15775485  0.10329004 -0.28960365  0.12862705 -0.1785271
  0.32189954 -0.09357474  0.04244087  0.24506603 -0.07425036  0.0458888  ]]
```

Figure 34: Les coefficients de la Régression Logistique

```
coefficients_df = pd.DataFrame(coefficients_lr, columns=X.columns)
coefficients_df = coefficients_df.transpose()
coefficients_df = coefficients_df.rename(columns={0: 'coefficients'})
sorted_coefficients_df = coefficients_df.sort_values(by=['coefficients'],key=lambda x: abs(x), ascending=False)
print(sorted_coefficients_df)
```

	coefficients
SLC7A5	-0.895275
RNF43	-0.814253
UGP2	0.596605
DAO	0.567863
NEURL1B	0.542553
NCAPH	-0.483930
DDIT4	-0.477950
FOXF2	0.470359
NIBAN1	0.456759
ADH1C	0.386067
PLAAT4	0.345840
EPS8L1	-0.328453
CLIC5	0.321900
DHRS11	0.309869
CPVL	0.301014
BSPRY	-0.289604
SOX18	0.272399
ANKRD40CL	0.263181
TRAPPC14	-0.261950
PAQR5	0.260356
TENTSC	0.250153
CRYL1	0.245066
KLF10	0.234833
TPM1	0.231441
PLA2G12B	0.230121

Figure 35: Classement des coefficients

Avant de sélectionner les trois gènes présentant les coefficients les plus élevés, il est essentiel de vérifier que ces gènes sont indépendants les uns des autres. En effet, si l'un de ces gènes est fortement corrélé ou dépendant d'un autre, sa sélection pourrait introduire une redondance inutile dans notre analyse. Dans ce cas, il serait plus judicieux d'exclure le gène redondant et de choisir le quatrième gène avec le coefficient le plus élevé, à condition qu'il soit indépendant des autres gènes déjà sélectionnés. Cette approche garantit que nous identifions les gènes qui apportent réellement une contribution unique et significative à notre modèle, sans biais dû à une multicollinéarité.

```
top_3_genes_data = data_final[top_3_genes.tolist() + ['tissue_status']]
sns.pairplot(top_3_genes_data, hue='tissue_status')
plt.show()
```

Figure 36: Visualisation des 3 gènes

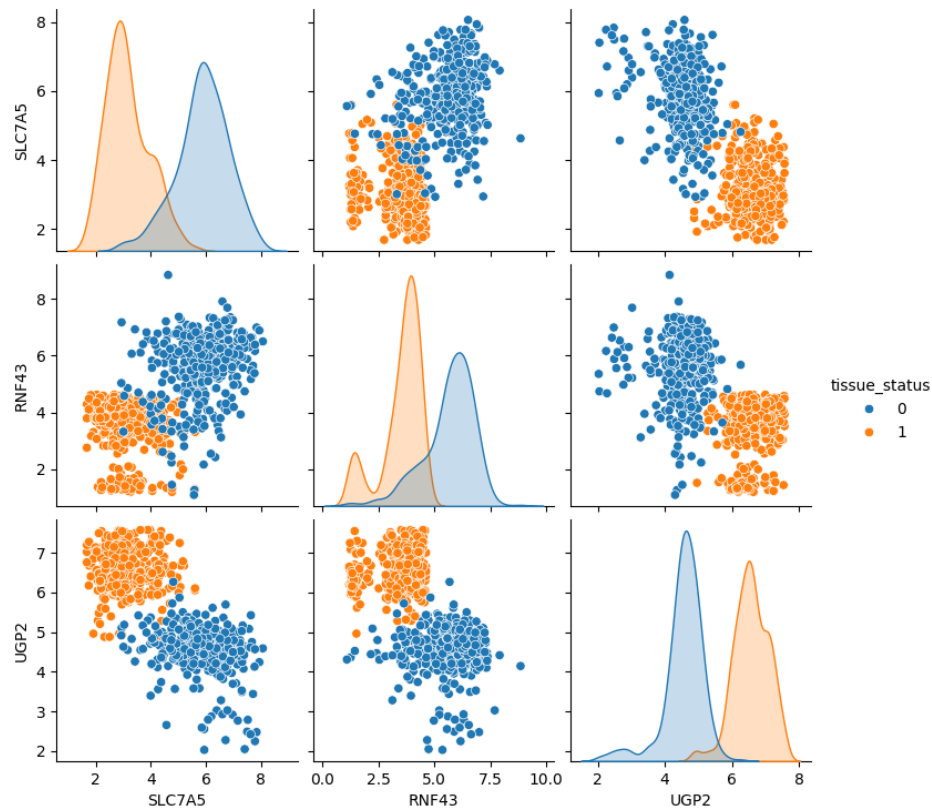


Figure 37: Corrélations entre les 3 gènes

Il est évident qu'il n'existe aucune forte corrélation entre les trois gènes sélectionnés, ce qui confirme leur indépendance les uns par rapport aux autres. Par conséquent, il est suffisant de se limiter à ces trois gènes pour notre analyse, sans avoir besoin d'en rechercher un quatrième. Cette sélection nous assure que chaque gène retenu apporte une information distincte et pertinente, sans redondance due à des relations de dépendance.

6 Evaluation de la performance des deux modèles en exploitant le top 1,2 et 3 des features.

Après avoir identifié les trois gènes ayant le plus d'impact sur les données, nous allons créer un nouveau tableau nommé **New_data**. Ce tableau contiendra uniquement les colonnes correspondant à ces trois gènes sélectionnés, ainsi que la colonne cible, **Tissue_status**. L'objectif de ce nouveau tableau est de simplifier notre analyse en nous concentrant uniquement sur les variables les plus significatives, tout en conservant l'information essentielle représentée par l'état des tissus.

```
top_3_features = sorted_coefficients_df.index[:3]
new_data = data[['tissue_status'] + list(top_3_features)]
print(new_data)
```

	tissue_status	SLC7A5	RNF43	UGP2
0	normal	3.864253	4.241267	7.062512
1	normal	3.069581	4.002000	6.318818
2	normal	3.188257	3.447174	6.917742
3	normal	2.357523	3.655234	7.081085
4	normal	4.137667	3.863294	5.937685
...	...	...	...	...
799	tumoral	5.947781	3.521571	4.393486
800	tumoral	4.746829	2.456133	4.794787
801	tumoral	5.916828	2.159680	4.438451
802	tumoral	5.694947	6.272607	4.822947
803	tumoral	3.844568	6.825288	4.985909

[804 rows x 4 columns]

Figure 38: New data

```
Y=new_data['tissue_status']
print(Y)
```

0	normal
1	normal
2	normal
3	normal
4	normal
...	...
799	tumoral
800	tumoral
801	tumoral
802	tumoral
803	tumoral

Name: tissue_status, Length: 804, dtype: object

```
X = new_data.drop('tissue_status', axis=1)
print(X)
```

	SLC7A5	RNF43	UGP2
0	3.864253	4.241267	7.062512
1	3.069581	4.002000	6.318818
2	3.188257	3.447174	6.917742
3	2.357523	3.655234	7.081085
4	4.137667	3.863294	5.937685
...	...	...	...
799	5.947781	3.521571	4.393486
800	4.746829	2.456133	4.794787
801	5.916828	2.159680	4.438451
802	5.694947	6.272607	4.822947
803	3.844568	6.825288	4.985909

[804 rows x 3 columns]

Figure 39: Séparage des features et de la colonne ciblée

```

X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2, random_state=42, stratify=Y)
scaler = StandardScaler()
scaler.fit(X_train)

X_train_scaled = scaler.transform(X_train)

X_test_scaled = scaler.transform(X_test)

print(f"Train set: {X_train_scaled.shape}, Test set: {X_test_scaled.shape}")

Train set: (643, 3), Test set: (161, 3)

```

Figure 40: Division et standardisation des données

```

y_test = y_test.astype('category').cat.codes
y_train = y_train.astype('category').cat.codes
model_lr = LogisticRegression()
model_lr.fit(X_train, y_train)
y_train_pred_lr = model_lr.predict(X_train)
y_test_pred_lr = model_lr.predict(X_test)
accuracy_train = accuracy_score(y_train, y_train_pred_lr)
print("Précision sur l'ensemble d'entraînement (Training):", accuracy_train)
accuracy_test = accuracy_score(y_test, y_test_pred_lr)
print("Précision sur l'ensemble de test:", accuracy_test)

Précision sur l'ensemble d'entraînement (Training): 0.9906687402799378
Précision sur l'ensemble de test: 0.9875776397515528

```

```

model_svm = SVC()
model_svm.fit(X_train, y_train)
y_train_pred_svm = model_svm.predict(X_train)
y_test_pred_svm = model_svm.predict(X_test)
accuracy_train_svm = accuracy_score(y_train, y_train_pred_svm)
print("Précision sur l'ensemble d'entraînement (Training) SVM :", accuracy_train_svm)
accuracy_test_svm = accuracy_score(y_test, y_test_pred_svm)
print("Précision sur l'ensemble de test SVM :", accuracy_test_svm)

Précision sur l'ensemble d'entraînement (Training) SVM : 0.9922239502332815
Précision sur l'ensemble de test SVM : 0.9875776397515528

```

Figure 41: Modèle SVM et Logistic regression appliqué sur notre new data

Les résultats montrent que le modèle de régression linéaire (RL) atteint une précision de 99.07% sur l'ensemble d'entraînement et de 98.76% sur l'ensemble de test, indiquant une bonne généralisation aux nouvelles données. Le modèle de machine à vecteurs de support (SVM) présente une précision légèrement supérieure sur l'ensemble d'entraînement à 99.22%, tout en maintenant la même précision de 98.76% sur l'ensemble de test, suggérant une performance comparable mais légèrement meilleure en apprentissage.

7 Comparaison entre les modèles appliqués sur colon_data et la nouvelle data.

Model	Data Type	Training Accuracy	Test Accuracy	Other Metric (e.g., F1-score)
SVM	Original Data	1	1	1
	New Data	0.9855	0.9801	0.9782
RL	Original Data	1	1	1
	New Data	0.9906	0.9875	0.9763

Table 1: Comparison of Metrics between SVM and RL on Original and New Data

Ce tableau compare les performances des modèles SVM et régression linéaire (RL) sur des données originales et des nouvelles données. Sur les données originales, les deux modèles (SVM et RL) atteignent une précision parfaite de 100% pour l'entraînement, le test et

l'autre métrique (par exemple, F1-score), indiquant qu'ils s'adaptent parfaitement aux données d'entraînement et de test. Cependant, sur les nouvelles données, la performance diminue légèrement pour les deux modèles : le SVM atteint une précision d'entraînement de 98.55% et de test de 98.01%, tandis que le RL atteint une précision d'entraînement de 99.06% et de test de 98.75%. Cela montre que le modèle RL conserve une précision légèrement supérieure à celle du SVM sur les nouvelles données, bien que la différence soit faible.

8 Case study

```
new_patient = {'RNF43': 4.68, 'SLC7A5': 4.10, 'DAO': 7.59}
new_patient_df = pd.DataFrame([new_patient])
features = model_lr.feature_names_in_
new_patient_df = new_patient_df.reindex(columns=features, fill_value=0)
new_patient_scaled = scaler.transform(new_patient_df)
pred_lr = model_lr.predict(new_patient_scaled)
proba_lr = model_lr.predict_proba(new_patient_scaled)
print(f"\nPrédiction pour le nouveau patient (Régression Logistique) : {pred_lr[0]}")
print(f"Probabilité : {proba_lr[0]}")

Prédiction pour le nouveau patient (Régression Logistique) : 1
Probabilité : [3.64400066e-10 1.00000000e+00]
```

Figure 42: Prédiction pour le nouveau patient

- **Est-ce que ce patient est atteint d'un cancer du côlon ?**

Oui, selon le modèle de régression logistique, le patient est prédit comme étant atteint d'un cancer du côlon. La prédiction est de "1", ce qui indique que le modèle classe ce patient comme appartenant à la classe "1", correspondant ici à un diagnostic de cancer du côlon.

- **Quelle est la probabilité prédite pour ce patient d'avoir un cancer du côlon ?**

La probabilité prédite pour ce patient d'avoir un cancer du côlon est de 1.00000000 (ou 100%). Cela signifie que le modèle est extrêmement certain de sa prédiction que ce patient a un cancer du côlon. La probabilité [3.64400066e-10, 1.00000000e+00] montre que la probabilité associée à ne pas avoir le cancer (classe 0) est presque nulle, tandis que la probabilité d'avoir le cancer (classe 1) est de 100%.

9 Conclusion

Dans ce projet, nous avons cherché à classer les échantillons en deux catégories distinctes, "normal" et "cancer", en utilisant des données d'expression génétique du cancer du côlon disponibles sur Kaggle. La première étape a consisté à préparer le dataset en téléchargeant les données, en les nettoyant, et en les divisant en ensembles d'entraînement et de test. Ce processus a inclus la gestion des valeurs manquantes et la standardisation des données pour garantir leur qualité.

L'exploration des données a permis d'analyser la distribution des classes et d'effectuer une analyse statistique descriptive des gènes. Cette étape a été cruciale pour comprendre

les caractéristiques des données et identifier les tendances importantes à l'aide d'outils de visualisation tels que les histogrammes et les diagrammes en boîte.

Nous avons ensuite appliqué plusieurs algorithmes de classification, y compris la régression logistique et les machines à vecteurs de support (SVM). Les performances de ces modèles ont été évaluées sur l'ensemble de test, montrant que les deux approches offraient des résultats solides. La régression logistique et le SVM ont montré une bonne précision, bien que des différences mineures aient été notées dans les performances des modèles.

L'évaluation des modèles a été réalisée en examinant les métriques pertinentes telles que la précision et la matrice de confusion. Ces outils ont permis de mieux comprendre les erreurs de classification et d'évaluer la capacité des modèles à généraliser sur de nouvelles données. L'analyse des gènes les plus importants a révélé leur impact sur les prédictions, fournissant des insights sur les caractéristiques les plus influentes dans la classification.

Dans une étude de cas, les mesures d'expression des gènes pour un nouveau patient ont indiqué une probabilité élevée d'avoir un cancer du côlon, avec une prédiction de 100% de la part des modèles. Cela a démontré la capacité des modèles à fournir des diagnostics fiables basés sur les données d'expression génétique.

Enfin, des tests supplémentaires ont été effectués avec des techniques comme k-Nearest Neighbors (k-NN) et les arbres de décision/forêts aléatoires. Ces analyses ont complété notre approche en affinant la sélection des paramètres et en identifiant les gènes les plus importants. En conclusion, les modèles appliqués ont montré une bonne performance dans la classification des échantillons, offrant des outils diagnostiques potentiels pour une utilisation clinique future.